

Longest Common Subsequence (LCS) Pattern Notes

Problem Statement

Given two sequences (arrays or strings), find the **maximum number of elements/characters that can be kept in the same order in both sequences**.

Example:

```
A = [1,2,3,4]
B = [2,3,5]

LCS = [2,3]
Length = 2
```

Why LCS is Important?

Many problems ask:

- Minimum insertions/deletions to make two strings equal
- Make two arrays identical
- Maximum common ordered elements
- Uncrossed lines
- Shortest common supersequence
- Edit distance variations

All of these are based on the same idea:

Find the maximum part that can remain unchanged.

That maximum unchanged part is the **Longest Common Subsequence (LCS)**.

What is a Subsequence?

A subsequence is obtained by:

- Picking some elements.
- Keeping their relative order.

Example:

```
A = [1,2,3]
```

Subsequences:

```
[]  
[1]  
[2]  
[3]  
[1,2]  
[1,3]  
[2,3]  
[1,2,3]
```

Not valid:

```
[2,1]  
[3,1]
```

because order changed.

What is a Common Subsequence?

```
A = [1,2,3,4]  
B = [2,3,5]
```

Common subsequences:

```
[]  
[2]  
[3]  
[2,3]
```

Longest one:

```
[2,3]
```

Length:

2

State Definition

Define:

$f(i, j)$

Meaning:

Length of LCS between:

$A[i:]$

and

$B[j:]$

Example:

$f(2, 3)$

means

LCS of

$A[2:]$

$B[3:]$

Recurrence Logic Building

Step 1: Base Case

If one sequence finishes:

```
if i == n or j == m:  
    return 0
```

Why?

Because:

$$\text{LCS}([], \text{anything}) = 0$$

No elements remain to match.

Step 2: Current Elements Match

Suppose:

$$A[i] == B[j]$$

Example:

$$2 == 2$$

Since both are equal, we should keep them.

So:

$$1 + f(i+1, j+1)$$

Why?

- We kept one element.
 - Solve the remaining suffixes.
-

Step 3: Current Elements Don't Match

Suppose:

$$A[i] != B[j]$$

Example:

$$1 != 2$$

At least one of them cannot be part of the LCS.

But which one?

We don't know.

So try both possibilities.

Choice 1

Skip A[i].

```
f(i+1, j)
```

Choice 2

Skip B[j].

```
f(i, j+1)
```

Take maximum.

```
max(  
    f(i+1,j),  
    f(i,j+1)  
)
```

Final Recurrence

```
if A[i] == B[j]:  
    return 1 + f(i+1,j+1)  
else:  
    return max(  
        f(i+1,j),  
        f(i,j+1)  
    )
```

Recursive Code

```
def f(i, j):  
    if i == n or j == m:  
        return 0  
  
    if A[i] == B[j]:  
        return 1 + f(i+1, j+1)  
  
    return max(  
        f(i+1, j),  
        f(i, j+1)  
    )
```

Answer:

`f(0,0)`

Dry Run

A = [1,2,3]
B = [2,3]

Start:

`f(0,0)`

Current:

`1 != 2`

Two choices:

`f(1,0)`
`f(0,1)`

`f(1,0)`

$$2 == 2$$

Return:

$$1 + f(2,1)$$

$$f(2,1)$$
$$3 == 3$$

Return:

$$1 + f(3,2)$$

$$f(3,2)$$

One sequence finished.

Return:

$$0$$

Therefore:

$$\begin{aligned} f(2,1) &= 1 \\ f(1,0) &= 2 \\ f(0,0) &= 2 \end{aligned}$$

Answer:

$$\text{LCS Length} = 2$$

Memoization

```
dp = [[-1]*(m+1) for _ in range(n+1)]

def f(i,j):

    if i == n or j == m:
        return 0

    if dp[i][j] != -1:
        return dp[i][j]

    if A[i] == B[j]:
        dp[i][j] = 1 + f(i+1,j+1)
    else:
        dp[i][j] = max(
            f(i+1,j),
            f(i,j+1)
        )

    return dp[i][j]
```

Time:

$O(n*m)$

Space:

$O(n*m)$

Tabulation

State

dp[i][j]

=
LCS of

A[i:]
and
B[j:]

Table Size

```
dp = [[0]*(m+1) for _ in range(n+1)]
```

Why $(n+1) \times (m+1)$?

Because:

```
i can become n  
j can become m
```

These represent empty suffixes.

Base Cases

```
dp[n][j] = 0  
dp[i][m] = 0
```

Already initialized with zeros.

Transition

If:

```
A[i] == B[j]
```

Then:

```
dp[i][j] = 1 + dp[i+1][j+1]
```

Else:

```
dp[i][j] = max(  
    dp[i+1][j],
```

```
    dp[i][j+1]  
)
```

How to Decide Loop Direction?

Look at dependencies.

```
dp[i][j]  
depends on  
  
dp[i+1][j]  
dp[i][j+1]  
dp[i+1][j+1]
```

Dependencies are:

- below
- right
- diagonal down-right

Therefore those cells must already be computed.

Hence:

```
Bottom → Top  
Right → Left
```

Loop Construction

```
for i in range(n-1, -1, -1):  
    for j in range(m-1, -1, -1):
```

Why?

Because:

When computing:

```
dp[i][j]
```

we need:

```
dp[i+1][j]
dp[i][j+1]
dp[i+1][j+1]
```

which are already available.

DP Table Visualization

```
      j→
      0 1 2 3
i  +-----
0  |
1  |
2  |
3  |
↓
```

Filling starts from:

bottom-right

and moves towards:

top-left

Final answer:

`dp[0][0]`

Tabulation Code

```
def lcs(A, B):
    n = len(A)
    m = len(B)
```

```
dp = [[0]*(m+1) for _ in range(n+1)]

for i in range(n-1,-1,-1):
    for j in range(m-1,-1,-1):

        if A[i] == B[j]:
            dp[i][j] = 1 + dp[i+1][j+1]
        else:
            dp[i][j] = max(
                dp[i+1][j],
                dp[i][j+1]
            )

return dp[0][0]
```

Space Optimization

Since only:

```
current row
next row
```

are needed,

space can be optimized to:

```
O(m)
```

using two arrays.

General Pattern Recognition

Whenever you see:

- maximum common ordered elements
- minimum insertions/deletions
- make two sequences equal
- order must remain same
- cannot rearrange
- choose or skip elements

Think:

LCS

Mental Template

Two sequences
↓
Need maximum common part
↓
Order matters
↓
LCS

Real World Interpretation

Suppose:

Old document:

ABCDXYZ

New document:

AXYZP

The unchanged part:

AXYZ

is the LCS.

Everything else:

- deleted
- inserted

This is how diff tools and version control systems work.

Minimum Insertions and Deletions Problem

Given:

```
A
B
```

Compute:

```
L = lcs(A,B)
```

Then:

```
deletions = len(A)-L
insertions = len(B)-L
answer = deletions + insertions
```

or

```
answer = len(A)+len(B)-2*L
```

Similar Problems

Easy

1. Longest Common Subsequence
2. Delete Operation for Two Strings
3. Minimum Insertions and Deletions

Medium

4. Uncrossed Lines
5. Shortest Common Supersequence
6. Minimum ASCII Delete Sum

Hard

7. Edit Distance

8. Distinct Subsequences
9. Interleaving String
10. Palindrome Partitioning variations

Pattern Identification Checklist

If the problem says:

- ✓ Two sequences
- ✓ Same order matters
- ✓ Maximum common part
- ✓ Minimum operations between two sequences
- ✓ Insert/Delete operations

Immediately think:

LCS Pattern

Master DP Template

```
def f(i,j):  
    if i == n or j == m:  
        return 0  
  
    if A[i] == B[j]:  
        return 1 + f(i+1,j+1)  
  
    return max(  
        f(i+1,j),  
        f(i,j+1)  
    )
```

This is one of the most important DP templates in interviews and competitive programming.